

TASK 1

• Java的八种基本数据类型是什么

byte, short, int, long, float, double, char, boolean

• 四种整型类型各占多少字节，表示范围多少？

类型	字节	范围
byte	1	-128~127
short	2	-32768~32767
int	4	-2147483648~2147483647
long	8	-9223372036854775808~9223372036854775807

• 基本类型与引用类型的区别

基本数据类型的变量储存的是真实的值，直接存放在对应的内存空间里，读取和修改都直接和这个值有关

引用类型的变量储存的是可以定位对应对象的一个地址，可以引用对象，但不是对象的本身

• 关于第一段代码

```
int a = 4;
char c = '0';
int b = a + c;
```

- 这里发生的转换：隐式转换，这里c是字符类型变量，在这个运算中被隐式转换为了整数类型
- b的值是多少？这里b=52，因为c这个'0'字符ASCII码是48，在运算中被转为int后c=48,a=4,48+4=52

• 第二段代码

```
int a = 5;
int b = 7;
int c = (++a) + (b++);
System.out.println(c);
System.out.println(a + " " + b);
```

- 输出结果

```
13
6 8
```

- 解释

```
int a = 5;
int b = 7; //定义变量a,b
int c = (++a) + (b++); //定义并赋值c, ++a是a+1之后的值, b++是b+1之前的值
System.out.println(c); //输出c, 6+7=13
System.out.println(a + " " + b); //输出a和b, 分别是加一过后的6和8
```

• 为什么两个正数相加可能变成负数？

在二进制存储中，是有存储数位的，以byte8位举例，为了表示符号，会选择最高位当做符号位，剩下7位表示数。

8位按十进制来说正数最大是七个一，01111111，所以最大正127

负数是用补码，最小10000000，也就是-128

如果127+2，是两个正数相加，但是结果超过了127，超出了byte可以表达的上限，也就是说溢出了，这里以二进制相加

```
01111111
+00000010
-----
10000001 /*这里可以看到最高位符号位变成了1，正常来说应该是0（不会变号），所以说溢出了，也就
是说这里变成了负数，由于负数使用补码，这个10000001表示-127*/
```

• 为什么float可能出现精度问题

float是单精度浮点数，有32位存储，其中分为符号位，尾数位和指数位

浮点数是用二进制的科学计数法来存储的，整数位默认是1，小数点之后的数字存入尾数位

但是在十进制转二进制下，可能出现无限的二进制数情况，由于尾数位有限，这时候只能省略掉后面的以进行后面的运算，这就导致了精度的问题

• 包装类与缓冲池

- 包装类：指可以包装8个基础数据类型值包装成对象的8个类，是一一对应的
- 缓冲池：缓冲池是包装类自带的静态缓存资源（Float，Double似乎没有），在对应值范围内，可以调用缓冲池仓库，直接查找值以复用地址，而不用创建一个新对象，节省内存提升性能

题目的代码

```
Integer x = new Integer(18);
Integer y = new Integer(18);
System.out.println(x == y);

Integer z = Integer.valueOf(18);
Integer k = Integer.valueOf(18);
System.out.println(z == k);

Integer m = Integer.valueOf(300);
Integer p = Integer.valueOf(300);
System.out.println(m == p);
```

这里有三个打印，**第一个是false，第二个是true，第三个又是false**。这里我一个一个说：

1. 第一段，这里对应int类型使用了Integer这个包装类，把int类型值18包装为对象，**这里分别new出来了x、y两个对象。**

在x==y的比较中（由于包装为对象了，这里比较的是地址），因为x、y是两个不同的对象，**地址不同**，输出false；

2. 第二段，这里也是用了这个包装类，与第一段的区别在于这里调用了Integer类里的valueOf方法，这个方法会直接进入缓冲池仓库查找范围内的值，范围对了就直接调用这个地址，**不创建新的对象了**，所以z和k的地址一样，true；
3. 第三段，这里除了数字大小，和第二段没有区别，但这正是最大的区别。缓冲池的**范围应该在-128~127**，这里valueOf查找了缓冲池，但是300太大了，缓存里找不到，于是转而**新建对象**。因此，**m、p是不同对象**，地址不同，false

TASK 2

- if-else 和 Switch-case

```
// 判断传入年份是否为闰年
boolean isLeapYear(int year) {
    if((year%4==0 && year%100!=0) || year%400==0 ){
        return true;
    }else{
        return false;
    }
}
```

- switch-case 和 if-else 的适用场景分别是什么？

- switch-case **更适用于case的值有限且较少时，基于一个变量的值根据不同的case输出结果**，它的条件判断能力并没有if语句那么强，但是它可以更方便地去对一个变量的不同值做出判断。**单出发点，多分支**
- if-else 语句**更适用于针对如表示范围这样更复杂的条件输出不同分支**，比Switchcase语句更加能针对多出发点等复杂条件的判断，适用范围更广一点

- switch-case 是 if-else 的语法糖吗？

我认为其实不是。

switch-case 确实在部分功能上更加简化了，但是和if-else还有许多语义上的不同点

首先是写法和功能，如果在case内不写break，会一直往下穿透，把下面的case都给输出了，if不需要这样跳出去（我感觉这一点最关键，这个方面可以引申出和if-else不同的用法）

然后是编译机制方面，switch-case语句在case密集的时候会定位到对应的case跳转，不像if-else会在每一条判断是否运行，这个行为和if-else出入也很大

• 循环（所需要的解释都在注释里了）

```
// 例如 n = 5 时，输出：
//      *
//     * *
//    *  *
//   *  *
//  *

void print(int n) {
    int i=0;    //i作为行号的辅助标记，我把图形主要分为上半和下半，上半区的行数递增，如n=5，则编号
0,1,2,1,0
    int j=0;    //j用于打印左边空格
    int k=0;    //k用于打印里面空格
    //第一个外for循环，进行行号的控制
    for(i=0;i<(n-1)/2;i++){
        //第二个内for循环，进行打印左边空格这个(n+1)/2-i-1表示的是不同行号需要左边空格的数量
        for(j=0;j<(n+1)/2-i-1;j++){
            System.out.print(" ");
        }
        //加入一个判断：区分顶点那个只有一颗星的情况
        if(i==0){
            System.out.print("*\n");
        }else{
            System.out.print("*");
            //第三个内for循环，进行打印里面空格，2*i-1表示的是不同行号需要里面空格的数量
            for(k=0;k<2*i-1;k++){
                System.out.print(" ");
            }
            System.out.print("*\n");
        }
    }

    //下半区的for循环和上面一样，依旧控制行号，我使用了递减，这样就不用重新写空格的循环了，因为相同行
号所需空格是一样的
    for(i=(n-1)/2;i>=0;i--){
        for(j=0;j<(n+1)/2-i-1;j++){
            System.out.print(" ");
        }
        if(i==0){
            System.out.print("*\n");
        }else{
            System.out.print("*");
            for(k=0;k<2*i-1;k++){
                System.out.print(" ");
            }
            System.out.print("*\n");
        }
    }
}

//篇幅有限，只写方法内内容了
```

• 递归和迭代(这个n我把它当做第n项了)

```
//递归
int fibonacci(int n) {
    //先判断是不是第一第二个项，这两个比较特殊直接为1
    if(n==1 || n==2){
        return 1;
    }else{
        return fibonacci(n-1)+fibonacci(n-2); //这个else我没有省略可能清晰一点，后面按通项公式
        一项一项调用自身方法
    }
}

//迭代
int fibonacci(int n) {
    //这里也一样，特殊项特殊处理
    if(n==1 || n==2){
        return 1;
    }else{
        //斐波那契数列核心就在于三个数，前两个相加得第三个结果
        int a=1;    //a就代表第一个数（是相对的第一个数，不是一直是1）
        int b=1;    //b是第二个
        int c=0;    //c是第三个，那个结果，最后返回的值就用c
        int i=3;    //由于反复迭代需要循环，这里借助i这个变量控制循环次数

        /*这里用for循环，让i从3开始加
        a, b分别是第一第二个加数，他们相加得到c
        随后往后推，原来的第二个数b成为了新的第一个数a
        同理，原来的第三个数c成为了新的第二个数b
        至于新的c，把新的a, b相加就可以得到了，以此类推直到我要的那个项*/
        for(i=3; i<=n; i++){
            c=a+b;
            a=b;
            b=c;
        }
        return c;
    }
}
```

◦ 递归和迭代的核心区别是什么？

核心区别在中间状态的处理。

递归的中间状态不保存在变量里面，而是保存在调用栈里。递归是反复调用自己这个方法，用以前方法的返回值代入新的方法得到最终返回值

迭代的中间状态就是保存在变量里的。在这个方法内的变量自身进行反复更新迭代，以得到最终值

◦ 为什么很多场景更偏好迭代？

主要原因是**性能更高，内存占用更小，不发生栈溢出**

- 迭代不用反复地调用方法并存入调用栈，不会导致栈溢出
- 且反复地存入调用栈也会产生不必要的内存占用，迭代的内存占用因此更小，速度也更快

- 迭代的循环中数值都是保存在自我迭代变量里的，这样debug的时候也更方便一点

什么时候更适合递归呢？

如果要遍历一个整体与子分支结构相同的树形结构，递归更省代码

如果要把一个大问题拆成几个小问题解决最后再合并（分治模型），递归也更好

• 循环是否总是能被递归取代？如果能，代价是什么？

递归理论上确实可以取代循环。递归一层层地调用自己，确实和循环可以做到等价的效果

但是每递归一层，就要压一次调用栈，这样循环次数大了之后会导致栈溢出

btw，在多层循环下，循环的可读性也比递归更好。而且循环有 `break` `continue` 这样的语句方便控制

我燃尽了那个汉诺塔写不出来www

汉诺塔这个算法我借鉴了b站的教程，这里就讲讲思路吧

汉诺塔递归的本质就在于把多盘数的问题转变为少盘数，

例如：

3个盘子，我首要目标就是把最底层的盘子放到目的地，但是它被压着，怎么办呢？

我们把三个柱子不分固定地分为起始柱、中间柱和目的地

转换一下思路：**要把最底下的挪到目的地，可以看做把除去最底下的其他部分塔挪到中间柱**

不论是三个还是四个还是n个，都是这样以此类推，依次递归减少

这样我们就可以把操作简化为以下两个：

1. 把n-1的盘子塔放在中间柱
2. 把第n个盘子从起始柱放到目的地（这个是我们实际的操作，要写出来）

接下来是代码的实现,由上面的思路可得，我们需要打印的移动操作只有从起始柱到目的地这一个

所以在hanoi方法里，参数分别为**盘子数，起始柱，中间柱(空余位置)，目的地**(这里abc分别代表起始中间目的地，但注意它们不是固定的)

```
public class HanoiPrint {
    void hanoi(int n, String a, String b, String c) {
        //n=1不需要递归了，直接操作
        if (n == 1) {
            System.out.println(a + "->" + c);
        } else {
            hanoi(n - 1, a, c, b);    //第一次我要把n-1个盘的塔挪到中间柱(空余位)，所以把b作为
            //我的目的地，a作为起始
            System.out.println(a + "->" + c); //挪第n个（最底层的）到目的地
            hanoi(n - 1, b, a, c); //由于上一次移动，b成为了起始点，我要继续把剩余的n-1塔目的
            //地，所以c作为我的目的地
        }
    }
}

//剩下的就在main把对象new出来打印就好啦
public static void main(String[] args) {
    HanoiPrint print = new HanoiPrint();
    print.hanoi(3, "A", "B", "C");
}
```

```
}
```

TASK 3

• Error和 Exception的区别是什么？面对它们时，程序的处理态度应该有什么不同？

- Exception是在程序执行时出现的**可处理的异常**，一般是代码出现了逻辑错误或外部条件变化等原因引起的
- Error主要是JVM、系统级别的底层故障，是程序无法恢复的错误
- 处理态度应该有什么不同？

- 关于Exception

Exception分为受检异常和非受检异常两种。

受检异常是编译器强制要求在代码中处理的，否则无法编译，应当**能处理就处理**

非受检异常本质大多是代码 bug，可以通过编译，应当**提前校验规避**，出现了应当**及时修复代码**，不要单纯捕获、掩盖bug

- 关于Error

出现了Error错误，一般说明运行环境已经不可靠了，应当**不尝试捕获、恢复等修复操作**，应记录**日志之后退出**

• checked exception 和 unchecked exception 的区别是什么？它们通常分别由什么原因引起？

受检异常 checked exception：继承Exception类，大多是**外部环境问题**，强制需要处理。例如文件不存在、网络连接中断、IO 读取失败。这类问题程序可以提前预期，具备处理的可能性

非受检异常 unchecked exception：继承RuntimeException类，大多是**代码逻辑缺陷**，编译器不会强制检查，可以通过编译。例如数组下标越界、除 0、类型转换错误。属于写代码考虑不周导致的 bug

• 题目：读取文件并计算平均值

ps：这里Average类里不构造方法了，感觉这样写注释不连贯(

而且我能力不足，目前只能想到写一个求平均数的方法，想不到还有啥方法可以设计了ww

以下是自定义异常类EmptyFileException:

```
//extends Exception继承Exception，表示这是一个异常类
public class EmptyFileException
extends Exception {

    //带参构造方法，创建异常对象时接收异常提示信息，参数传入错误描述的字符串
    public EmptyFileException(String message) {
        //super调用父类保存异常信息，getMessage()可以取出这个字符串
        super(message);
    }
}
```

以下是类Average:

```
import java.io.BufferedReader; //这个类用于提供"readLine"方法以按行读取文件
import java.io.FileReader; //这个类用于读取文件，建立和文件的连接
import java.io.FileNotFoundException; //这个类用于处理文件未找到异常
import java.io.IOException; //这个类用于处理输入输出异常

public class Average {
    public static void main(String[] args) {

        /*用try-with-resources语句打开文件并用完后自动关闭
        其中，new FileReader用来打开当前目录的data.txt，建立和文件的连接
        FileReader是字符输入流，直接对接磁盘文件，但是没有readLine能力
        br变量保存BufferedReader对象的地址，方便通过它找到这个对象以调用readLine方法
        后续try也会用这个br变量自动调用br.close()方法关闭文件
        */
        try(BufferedReader br=new BufferedReader(new FileReader("data.txt"))){
            String line; //先声明一个字符串变量line用来存放每一行的文字
            int sum=0; //声明一个整数变量sum用来存放总和
            int count=0; //声明一个整数变量count用来存放有效整数的个数

            /*使用while循环读取每一行文字，直到文件末尾
            循环条件：1.br.readLine读取文件的一行文字，返回字符串，到末尾返回null
                     2.把br.readLine()的结果赋值给line，如果line不为空，则继续循环
            */
            while((line=br.readLine())!=null){

                /*一个条件判断line.isBlank(),如果遇到空行会直接重新开始循环读下一行
                用来避免空行导致NumberFormatException*/
                if(line.isBlank()){
                    continue;
                }

                //平均数计算的核心部分
                /*Integer.parseInt这个方法可以把字符串的数字转为整数
                如果因为有其他字符导致无法转为整数，会抛出NumberFormatException
                */
                int num=Integer.parseInt(line);
                sum+=num; //把num加到sum上
                count++; //有效整数个数加一
            }
        }
    }
}
```

```

//条件判断，防止无有效整数使count=0，导致除以零错误
if (count==0){

    //如果是空文件，这里把这个自定义的空文件错误throw出去，后面可以catch到
    throw new EmptyFileException("无有效整数");
}

double average=(double)sum/count;//计算平均数，把sum转为double，否则结果会是int
System.out.println("平均值为: "+average);

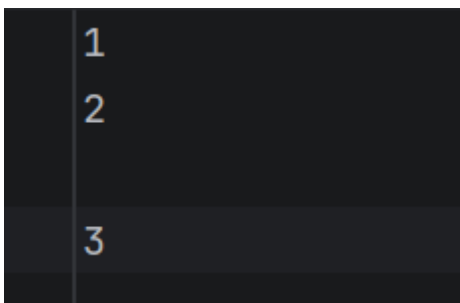
/*以下是catch部分，处理可能的异常
1.FileNotFoundException 处理文件未找到异常
2.NumberFormatException 处理文件内有异常字符的异常
3.EmptyFileException 处理空文件异常,e.getMessage可以取出上面throw出来的报错字符串
4.IOException 处理文件损坏等其他IO异常
*/
}catch(FileNotFoundException e){
    System.out.println("找不到data.txt");
}catch(NumberFormatException e){
    System.out.println("文件中包含无法解析为整数的内容");
}catch (EmptyFileException e){
    System.out.println("错误:"+e.getMessage());
}catch (IOException e){
    System.out.println("读取文件时发生错误");
}
}
}

```

• 一些小问题

- 1.开始我一直都是找不到data.txt，后面ai大人告诉我打印System.getProperty("user.dir")找到文件夹才发现文件夹放错了(
- 2.IOException我开始以为不需要catch了，因为题目里没要求报这个错误，但是没有了之后编译不通过.....原来readLine方法会抛出IOException，必须要么catch它，要么main里又把它throw了

结果截图（只放两个情况了）



1
2
3

← data.txt_2

← data.txt_1



1
2
UWU
3

平均值为: 2.0

➡ 输出结果1

D:\soft\OracleJDK\jdk25\
文件中包含无法解析为整数的内容

➡ 输出

结果2

TASK 4

题目代码

```
List<String> strings = List.of("I", "am", "a", "list", "of", "Strings");  
Stream<String> stream = strings.stream();  
Stream<String> limit = stream.limit(4);  
System.out.println("limit = " + limit);
```

• 直接打印 limit，会得到什么类型的结果？

我输出了 `java.util.stream.SliceOps$1@3f99bd52`，这个是字符串类型的结果：类名+@+十六进制哈希码

• 为什么不会直接把流里的元素打出来？

1. 这段代码获取了stream流，但是没有写终结方法，这个stream.limit(4)中间方法没有开始执行
2. 就算写了终结方法，执行了这个中间方法，这个limit也是被stream流对象赋值了，输出结果依旧是上面那一堆字符串

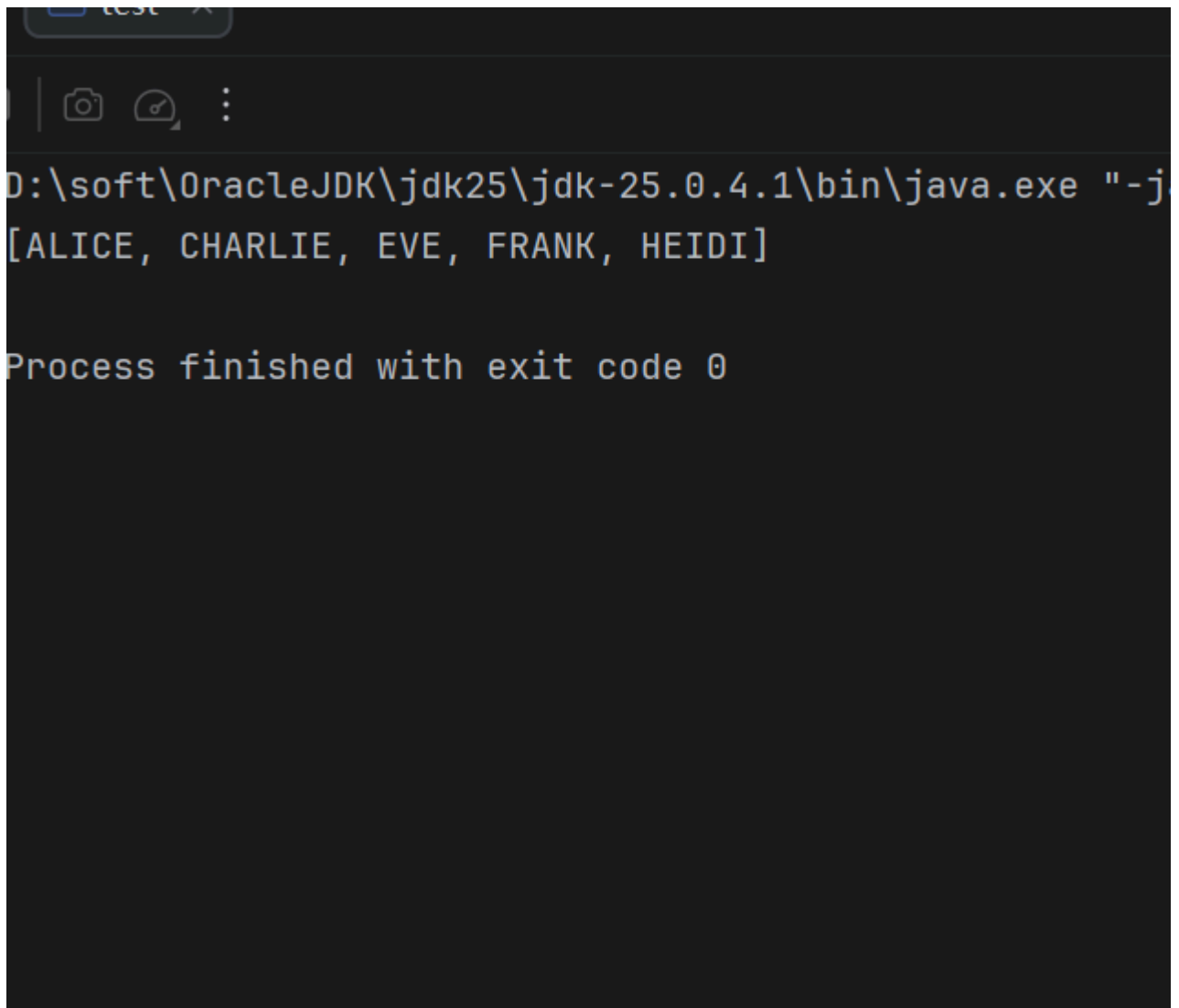
要直接打印出来可以用forEach遍历元素

• 最后一题

```
public class Student {  
    String name;  
    int score;  
  
    public Student(String name, int score) {  
        this.name = name;  
        this.score = score;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public int getScore() {  
        return score;  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        List<Student> students = Arrays.asList(  
            new Student("Alice", 85),  
            new Student("Bob", 58),  
            new Student("Charlie", 90),  
            new Student("David", 45),  
            new Student("Eve", 72),  
            new Student("Frank", 60),  
            new Student("Grace", 55),  
            new Student("Heidi", 95)  
        );  
  
        List<String> passingStudents = students.stream()  
            //中间方法filter, 只保留符合后面条件的元素  
            .filter(student -> student.getScore() >= 60)  
            //中间方法map, 把学生的名字改成大写  
            .map(student -> student.getName().toUpperCase())  
            //sorted, 按自然顺序也就是名字字母顺序排序  
            .sorted()  
            //终结方法collect, toList方法把流里的元素输出在一个新的list, 赋值给passingStudents  
            .collect(Collectors.toList());  
  
        System.out.println(passingStudents);  
    }  
}
```

运行结果截图



A screenshot of a Windows command prompt window. The title bar at the top shows a tab labeled "test". Below the title bar is a menu bar with icons for a camera, a speech bubble, and a vertical ellipsis. The command prompt shows the following text:

```
D:\soft\OracleJDK\jdk25\jdk-25.0.4.1\bin\java.exe "-j  
[ALICE, CHARLIE, EVE, FRANK, HEIDI]  
  
Process finished with exit code 0
```

终于结束了，第二题怎么比第一题难度跨越这么大啊